

UNIT 3

Instruction Types: Basically instruction can be classified into two categories: Memory referenced Instruction & Register referenced Instruction:-

Memory-Referenced Instructions:

Memory-referenced instructions are those that operate on data stored in memory locations. In these instructions, one or more memory addresses are specified as operands, and the CPU accesses the data from these memory addresses to perform the operation. Memory-referenced instructions are common in architectures that use a load-store model, where data must be explicitly loaded from memory into registers before it can be operated on.

Example memory-referenced instructions:

- **LOAD R1, [A]:** Load the value stored at memory address **A** into register **R1**.
- **STORE [B], R2:** Store the value in register **R2** into memory address **B**.
- **ADD R3, [X], [Y]:** Add the values stored at memory addresses **X** and **Y** and store the result in register **R3**.

Register-Referenced Instructions:

Register-referenced instructions are those that operate directly on registers without accessing memory. In these instructions, all operands are registers, and the CPU performs the operation using the values stored in the specified registers. Register-referenced instructions are common in architectures that use a register-based model, where most operations are performed within registers.

Example register-referenced instructions:

- **ADD R1, R2, R3 :** Add the values in registers **R2** and **R3** and store the result in register **R1**.
- **SUB R4, R5, R6 :** Subtract the value in register **R6** from the value in register **R5** & store the result in register **R4**.
- **AND R7, R8, R9 :** Perform bitwise AND operation between the values in registers **R8** and **R9** & store the result in register **R7**.

Instruction Formats (Zero, One, Two and Three Address Instruction)

An instruction is of various lengths depending upon the number of addresses it contains. Generally CPU organization is of three types on the basis of number of address fields:

1. Single Accumulator organization (One address Instruction)
2. General register organization (Two or Three address instruction)
3. Stack organization (Zero address Instruction)

- **Three address Instruction codes for Expression:**

$$X = (A+B)*(C+D)$$

ADD	R1, A, B	$R1 = M[A] + M[B]$
ADD	R2, C, D	$R2 = M[C] + M[D]$
MUL	X, R1, R2	$M[X] = R1 * R2$

Here ,

R1, R2 are registers

M[A] is any memory at address location A

- **Two address Instruction codes for Expression:**

$$X = (A+B)*(C+D)$$

MOV	R1, A	$R1 = M[A]$
ADD	R1, B	$R1 = R1 + M[B]$
MOV	R2, C	$R2 = M[C]$
ADD	R2, D	$R2 = R2 + M[D]$
MUL	R1, R2	$R1 = R1 * R2$
MOV	X, R1	$M[X] = R1$

- **One address Instruction codes for Expression:**

$$X = (A+B)*(C+D)$$

LOAD	A	$AC = M[A]$
ADD	B	$AC = AC + M[B]$
STORE	T	$M[T] = AC$
LOAD	C	$AC = M[C]$
ADD	D	$AC = AC + M[D]$
MUL	T	$AC = AC * M[T]$
STORE	X	$M[X] = AC$

- **Zero address Instruction codes for Expression:**

$$X = (A+B)*(C+D)$$

Postfixed :- $AB+CD+*$

PUSH	A	$TOP = M[A]$
PUSH	B	$TOP = M[B]$
ADD		$TOP = M[A] + M[B]$
PUSH	C	$TOP = M[C]$
PUSH	D	$TOP = M[D]$
ADD		$TOP = M[C] + M[D]$
MUL		$TOP = (M[C] + M[D]) * (M[A] + M[B])$
POP	X	$M[X] = TOP$

Instruction Cycle :

The instruction cycle, also known as the fetch-execute cycle, is a fundamental concept in computer architecture that describes the process by which a computer retrieves a program instruction from memory, decodes it, executes it, and then stores the results. It's a continuous cycle that forms the basis of how a CPU (Central Processing Unit) processes instructions.

Here are the basic steps of the instruction cycle:

1. **Fetch:** The CPU retrieves (or "fetches") the next instruction from the computer's memory. It locates the instruction using the program counter, which points to the memory address of the next instruction.
2. **Decode:** The fetched instruction is then decoded. This step involves determining what operation the instruction is specifying and what data it needs to work with. The CPU interprets the instruction to understand what needs to be done.
3. **Execute:** The CPU performs the action or operations specified by the decoded instruction. This might involve arithmetic or logical operations.
4. **Store:** If the instruction results in a change in data, the CPU will store the results back into the appropriate place in memory or registers.

Once the execution of an instruction is completed, the cycle continues, and the CPU fetches the next instruction based on the updated program counter.

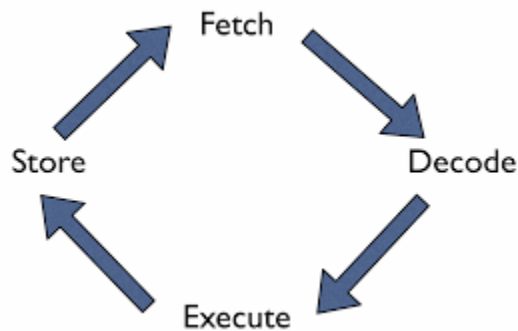
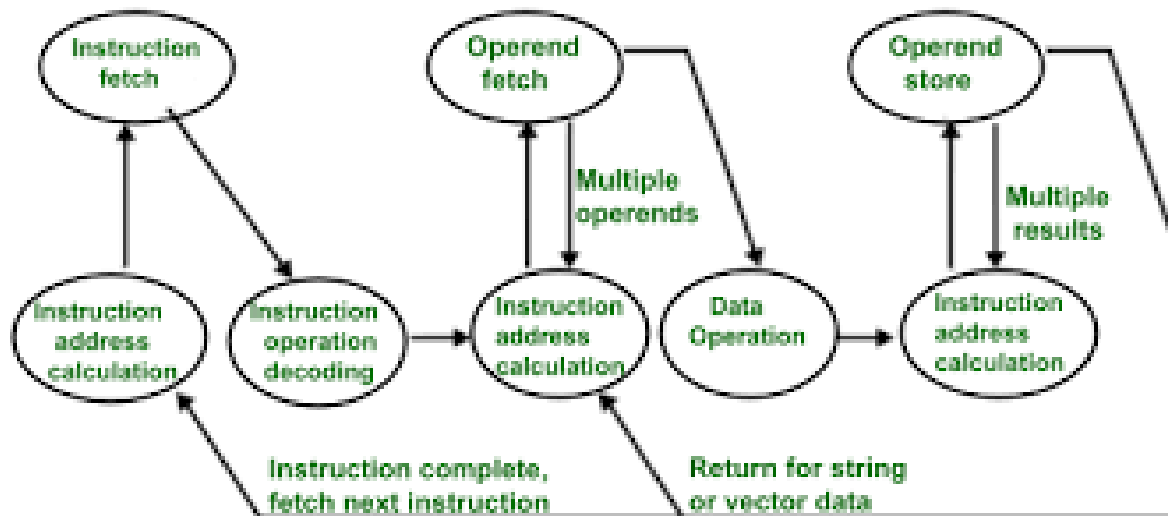


Fig: A Typical Instruction Cycle

Execution of a complete instruction

The execution of a complete instruction involves several stages



1. **Instruction Address Calculation:** PC (Program Counter) is a register which contains the next instruction to be executed. This address is then transferred from PC to MAR (Memory Address Register).
2. **Instruction Fetch:** The CPU fetches the instruction from memory at address specified by MAR. In parallel 'READ' control signal is enable. The instruction is then in placed in register IR (Instruction Register). This transfer occurs over the BUS.
3. **Instruction Decode:** The fetched instruction is decoded to determine the operation to be performed and the operands involved. This stage involves breaking down the instruction into its opcode (operation code) and identifying the addressing mode (how to access the operands).
4. **Operand Fetch:** If the instruction involves fetching data from memory or registers, the CPU retrieves the necessary operands. This might involve accessing data from registers, cache, or main memory, depending on the addressing mode specified in the instruction.

5. **Execution:** The CPU performs the specified operation on the fetched operands. This could involve arithmetic or logic operations, data movement, or control transfer operations, depending on the instruction.
6. **Write-back:** If the instruction produces a result that needs to be stored, the CPU writes the result back to the destination location, which could be a register, memory location, or an I/O device.

Micro-operation:- A **micro-operation** (also called **micro-op**, μop) is an elementary operation performed on data stored in registers (or buses) inside the CPU.

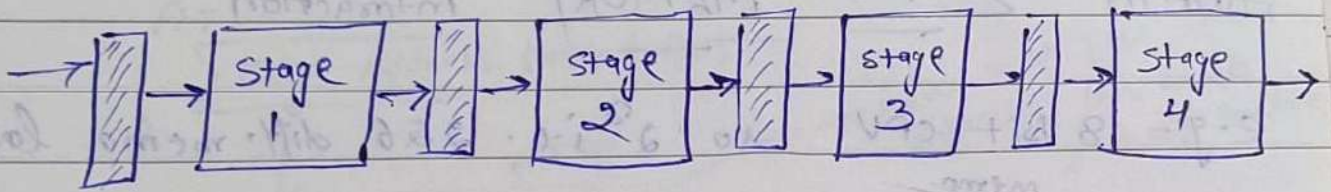
Like

- Moving data from one register to another
- Adding two register values
- Performing a bitwise logic operation
- Shifting bits within a register

Cycles →	1	2	3	4	5	6	7	8	9
Instr									
I1	[IF]	[ID]	[EX]	[WB]					
I2		[IF]	[ID]	[EX]	[WB]				
I3			[IF]	[ID]	[EX]	[WB]			
I4				[IF]	[ID]	[EX]	[WB]		
I5					[IF]	[ID]	[EX]	[WB]	
I6						[IF]	[ID]	[EX]	[WB]

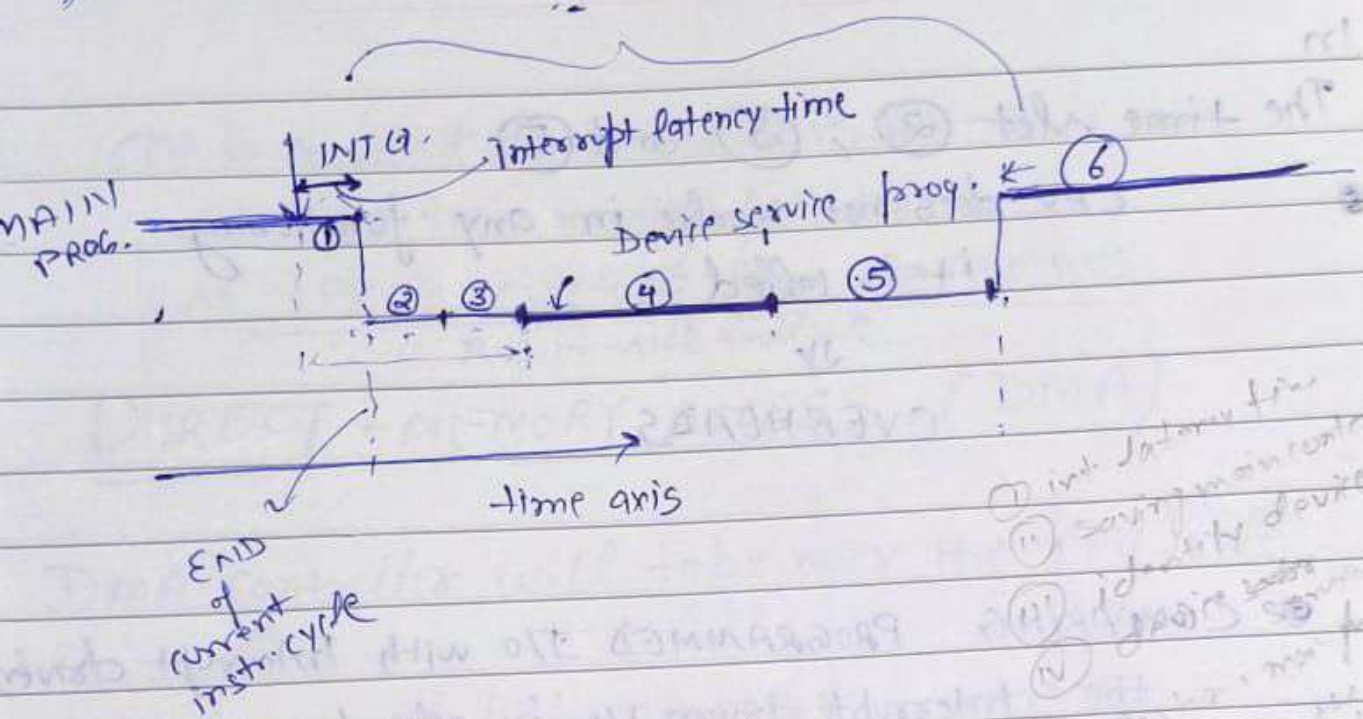
Instruction Pipelining

An ideal Pipeline



- All objects go through the same stages
- No sharing of resources between any two stages
- Propagation (time) delay through all pipeline's stages is equal
- Scheduling of a transaction entering the pipeline is not affected by the transactions in other stages.

But 1 instruction actually depend on earlier instruction
So instructions depend on each other ~~causing~~ causing
(various hazards.)

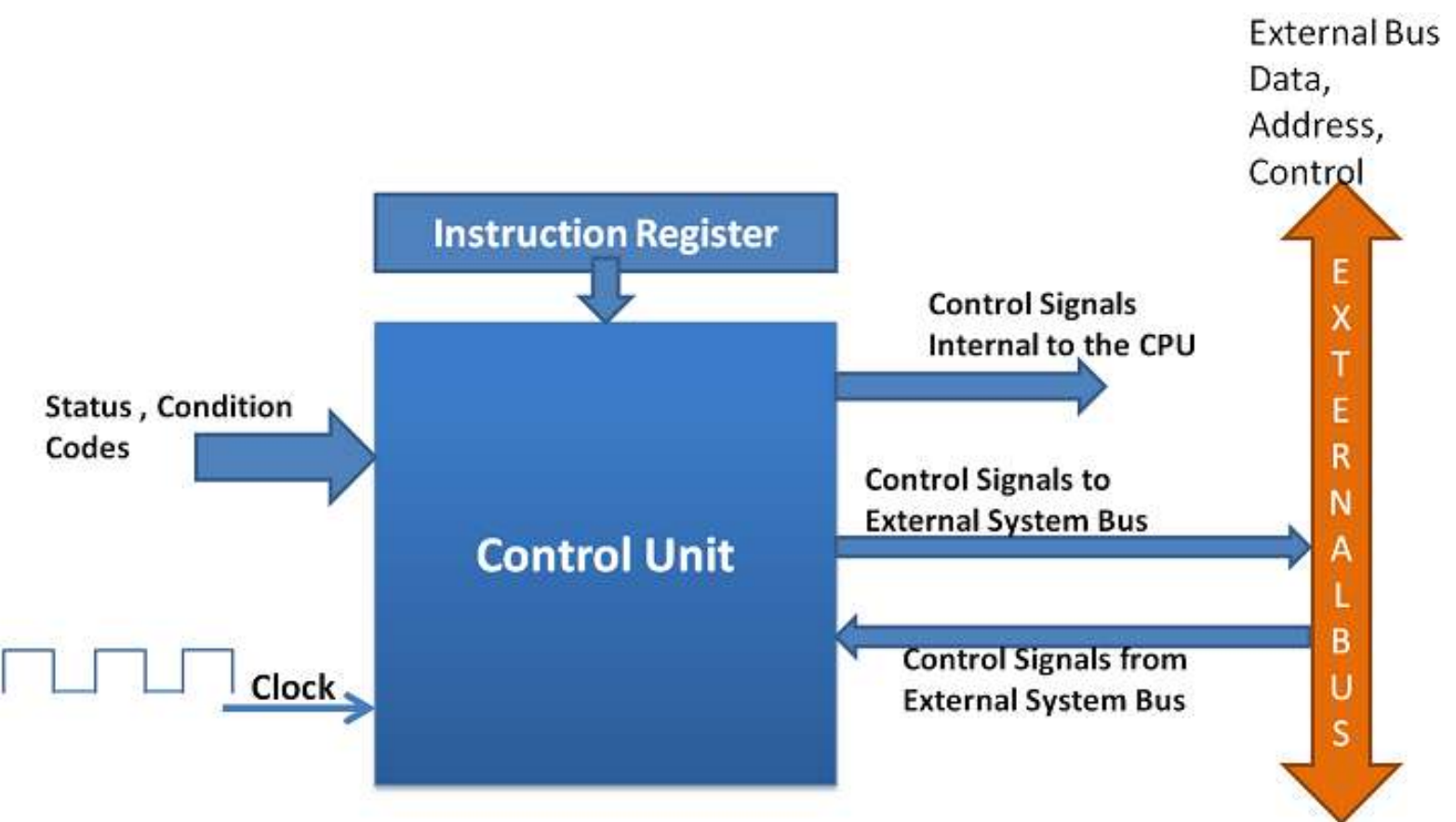


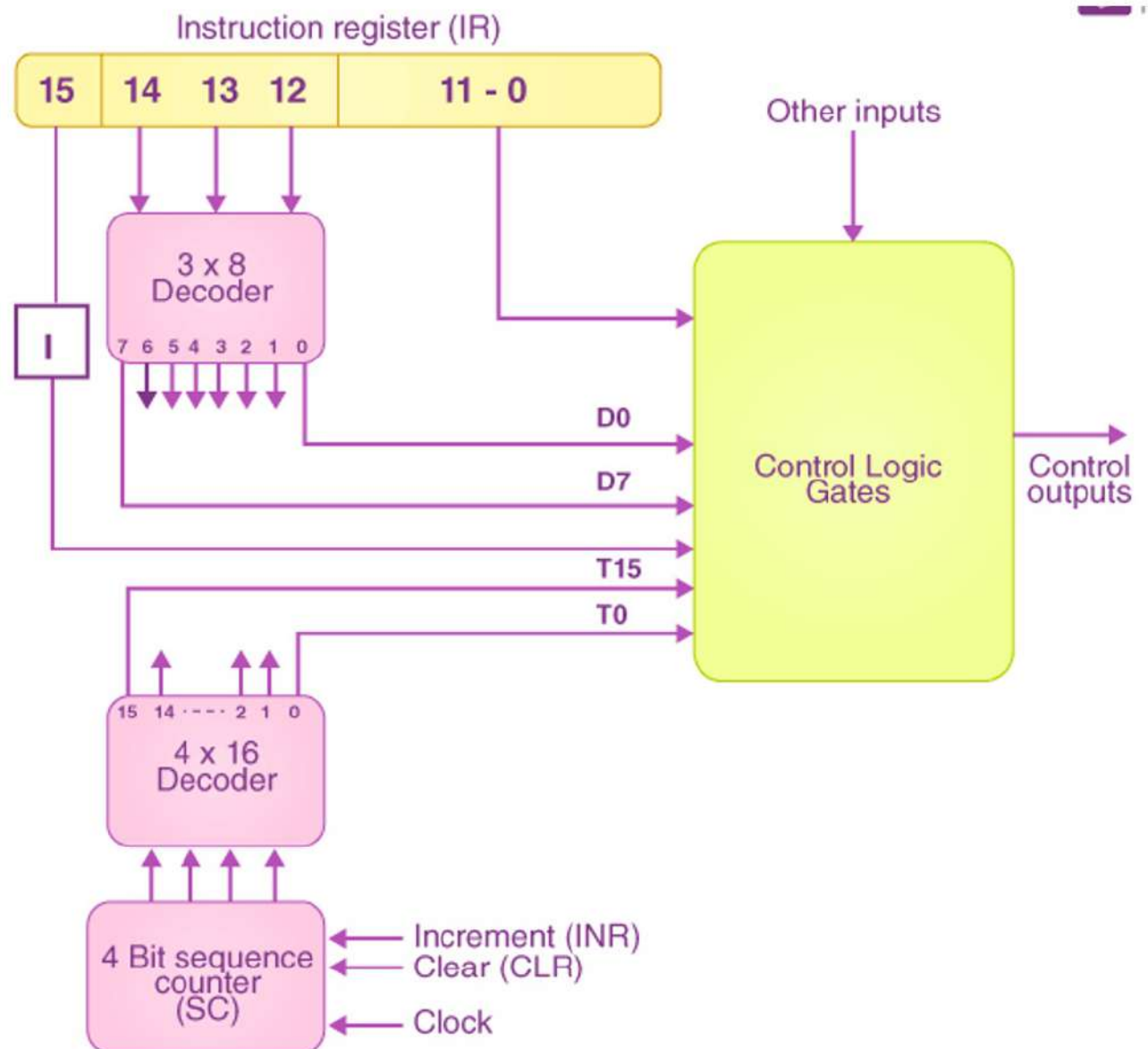
- ① - Interrupt latency time
- ② - SAVING MAIN CONTEXT
- ③ - IDENTIFYING DEVICE
- ④ - INTERRUPT SERVICE (running a subroutine)
- ⑤ - RESTORING MAIN CONTEXT
- ⑥ - RECEIVING the main program.

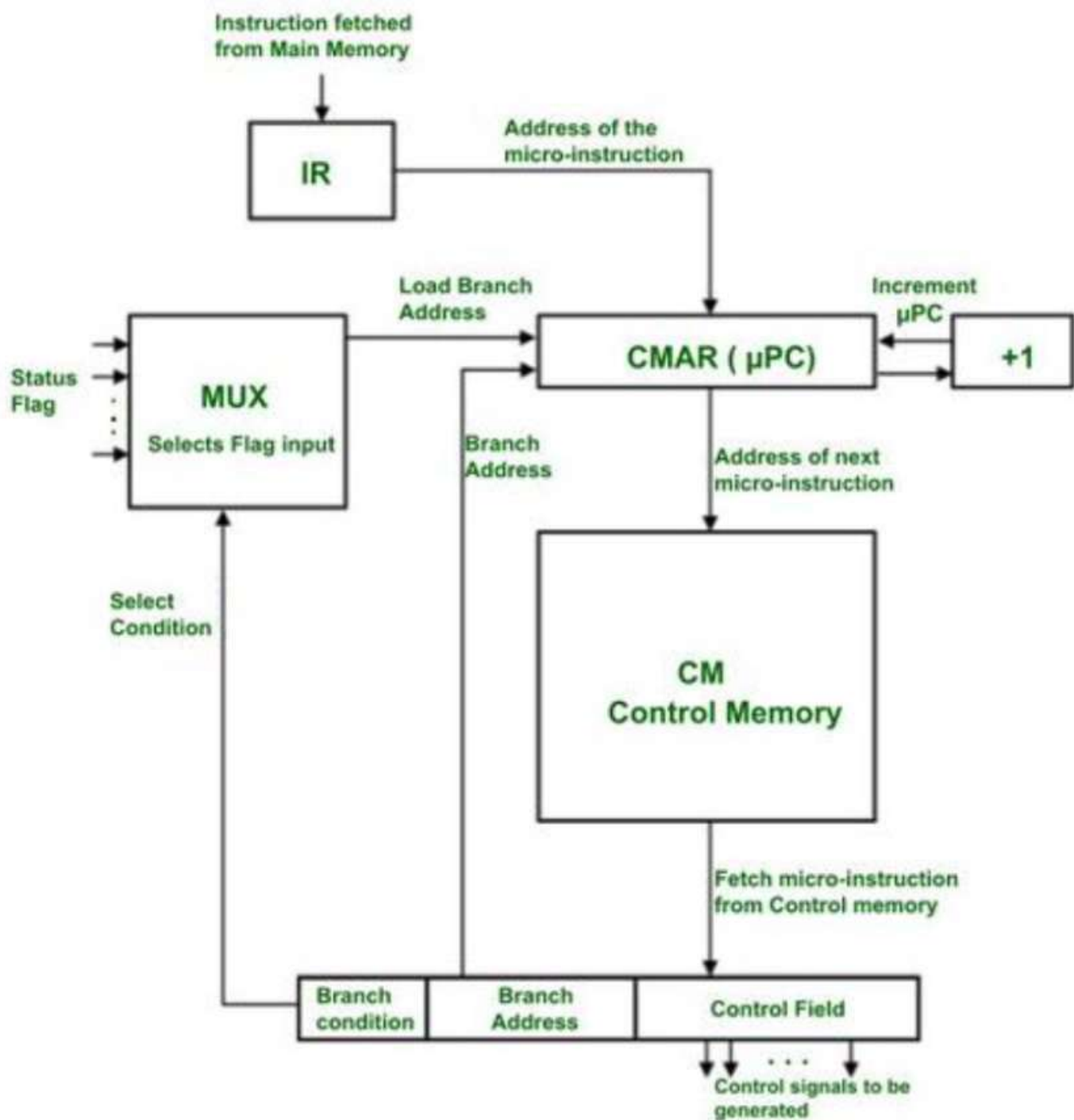
Note:- At time when device is generating INT and is identifying itself with some identification code then we say it

at the time of interrupt req. (VECTORED INTERRUPT)
it is known the start addr. of device service prog.

(NON VECTORED INTERRUPT)
at the time of INT req.
it is not known which is the actual req's start address of device service prog.







Control Unit :- [CU]

The basic function of a processor is to execute an instruction completely which involves a sequence of substeps called cycles. Each cycle is made up of a sequence of more fundamental operations, called micro-operations. A single micro-operation involves a transfer between registers, and an external bus or a simple ALU operation.

The CU is responsible for handling such micro-operation. It performs two tasks -

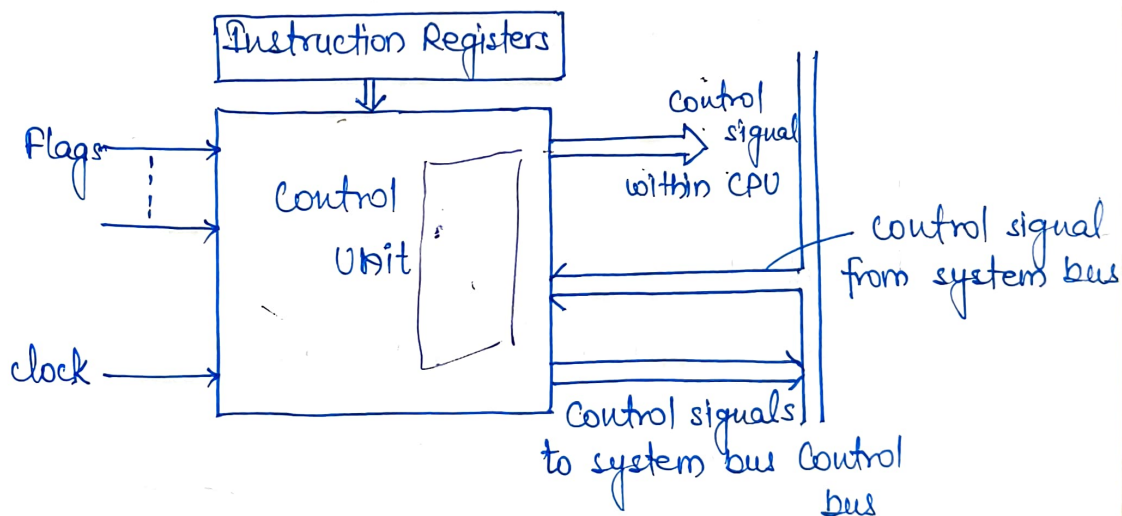
- 1.) It causes the processor to execute micro-operations in the proper sequence.
- 2.) It generates the control signals that cause each micro-operation to be executed.

NOTE:- These control signals cause the opening and closing of logic gates that results in the transfer of data to and from registers and the operations of the ALU.

There are 2 techniques for implementing a CU -

- i) Hardwired Control
- ii) Micro-program control.

~~General model of the CU~~ :- General model of the CU



The input to the CU are -

- i) Instruction Registers - The opcode of the current instruction determines the micro-operation

to be performed during the execute cycle.

2. Flags - The CU determines the states of the processors and the outcome of the previous ALU operations by flags.

3. Control Signals from system Bus - The control bus portion of the system bus provides signals to the CU such ^{as} ~~that~~ interrupt signals and acknowledgement etc. $\rightarrow$

Functional Requirements for Controlling :-

Functional requirements for CU are those functions that the CU must perform. These functional requirements are the basis for design and implementation of the CU.

The program execution consists of operations that involve following processor elements -

- ALU
- Registers
- Internal data path
- External data path (System Bus)

We can say that the CU has to perform the following two tasks -

1. Sequencing :- Based on program sequence the CU instructs the processor to step through a

series of micro-operations in the proper sequence.

2. Execution :- The CU causes each micro-operation to be performed.

The output from the CU are as follows -

1. Control signals within the processor -

There are further 2 types of signals. -

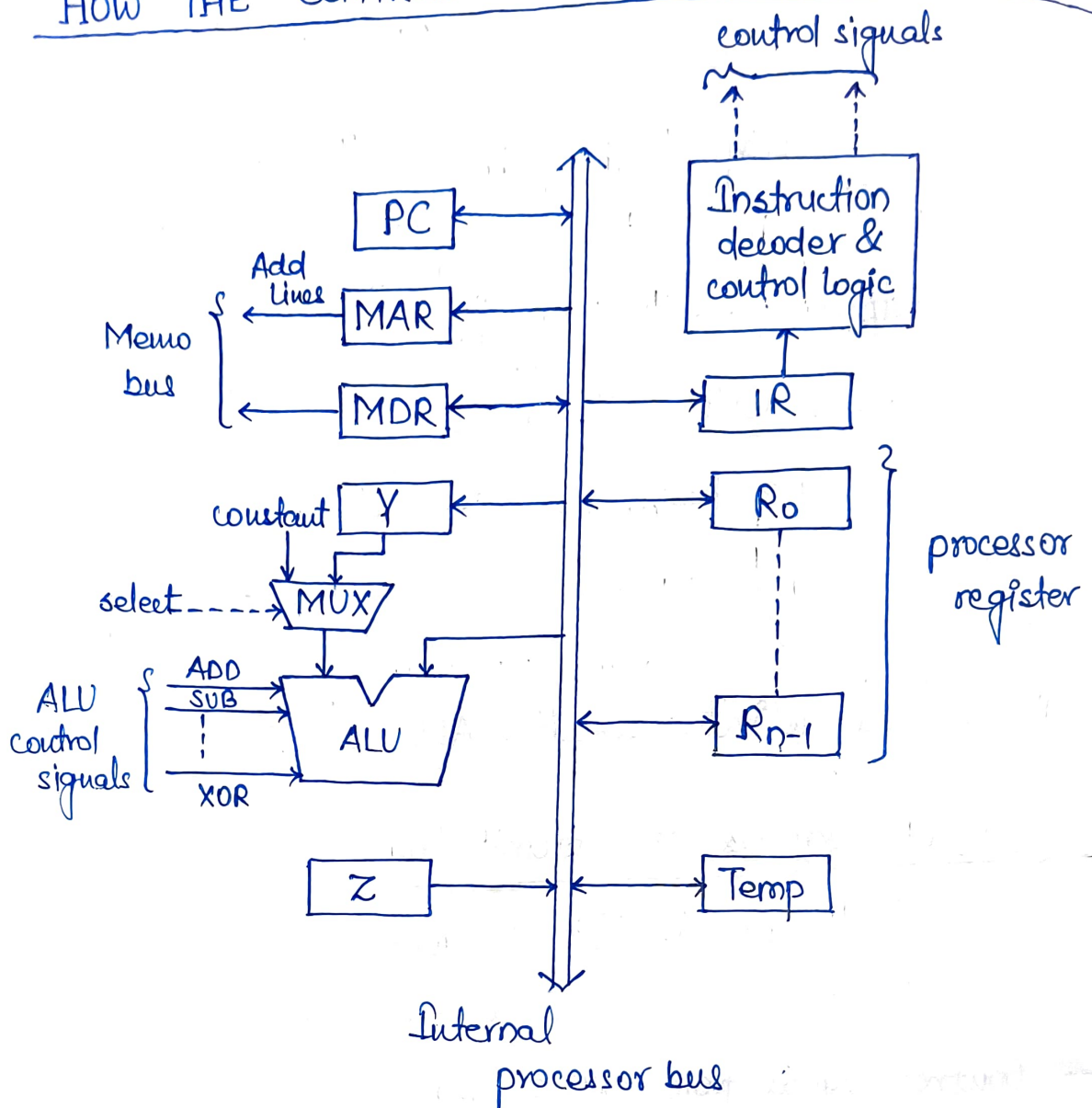
- Those that are responsible for data transfer from one register to another.
- Those that activate specific ALU operations.

2. Control Signals to control bus -

These are of also 2 types -

- Control signals from CPU to memory.
- Control signal from CPU to input/output module.

HOW THE CONTROL UNIT MAINTAINS THE CONTROL



Z → General purpose register

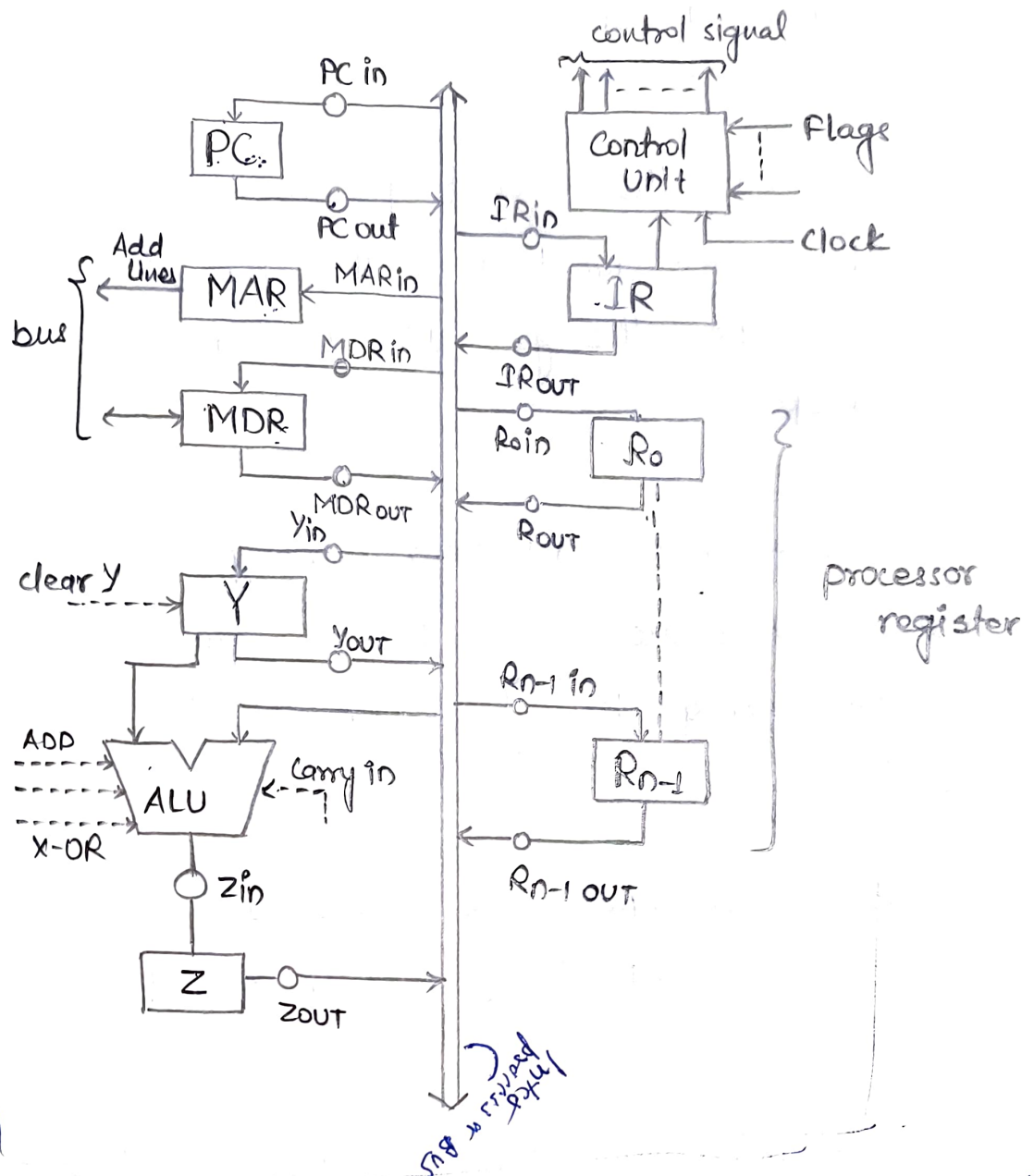
IR → Instruction register

Temp → Temporary register

MAR → Memory Addr Register

PC → Program Counter

MDR → Memory Data Register



Some Example of control steps for diffⁿ types of funⁿ

1. Signals for transferring the content of register R_0 to register R_1 [$R_1 \leftarrow R_0$]

$$\text{Set } R_{0\text{out}}, R_{1\text{in}} = 1$$

2. Signals for reading a memory location, add^r in register R_3 .

$$\text{Set } R_{3\text{out}}, \text{MAR}_{\text{in}}, \text{Read} = 1$$

Register Transfer

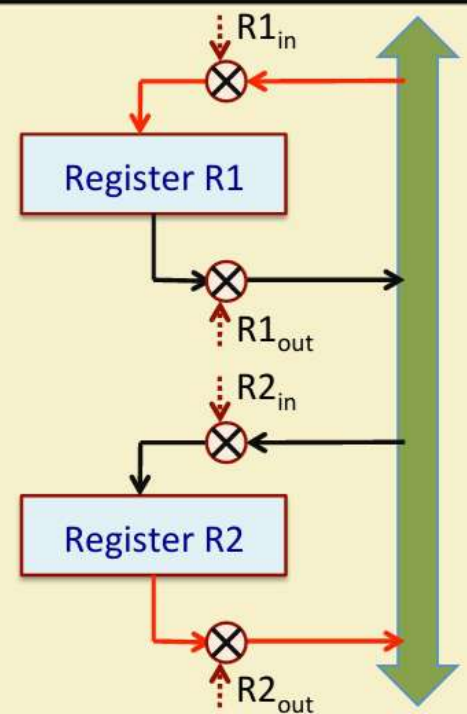
MOVE R1, R2 // $R1 \leftarrow R2$

- Enable the output of R2 by setting $R2_{out} = 1$.
- Enable the input of register R1 by setting $R1_{in} = 1$.
- All operations are performed in synchronism with the processor clock.
 - The control signals are asserted at the start of the clock cycle.
 - After data transfer the control signals will return to 0.

– We write as $T1: R2_{out}, R1_{in}$

Time Step

Control Signals



ALU Operation

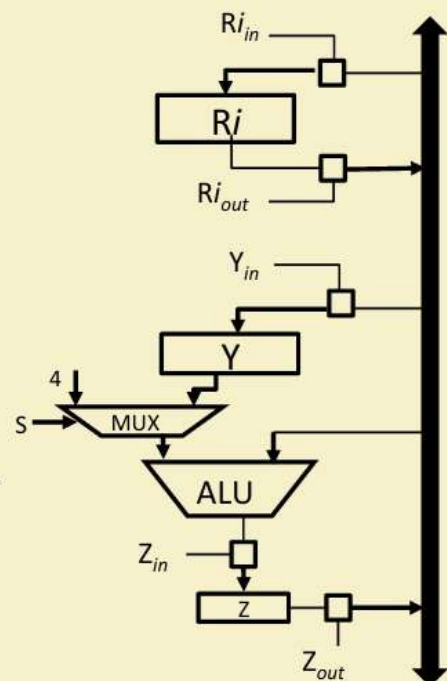
ADD R1, R2 // R1=R1 + R2

- Bring the two operands (R1 and R2) to the two inputs of the ALU.
One through Y (R1) and another (R2) directly from internal bus.
- Result is stored in Z and finally transferred to R1.

T1: R1_{out}, Y_{in}

T2: R2_{out}, SelectY, ADD, Z_{in}

T3: Z_{out}, R1_{in}



Fetch a word: **MOVE R1, (R2)**

1. $MAR \leftarrow R2$
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory
5. $R1 \leftarrow MDR$

Control steps:

- a) $R2_{out}, MAR_{in}, \text{Read}$
- b) MDR_{inE}, WMFC
- c) $MDR_{out}, R1_{in}$

Store a word: **MOVE (R1), R2**

1. $MAR \leftarrow R1$
2. $MDR \leftarrow R2$
3. Start a Write operation on the memory bus
4. Wait for the MFC response from the memory

Control steps:

- a) $R1_{out}, MAR_{in}$
- b) $R2_{out}, MDR_{in}, Write$
- c) $MDR_{outE}, WMFC$

Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

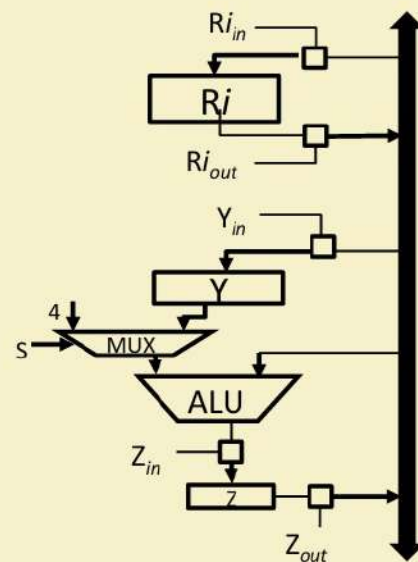
T2: Z_{out} , PC_{in} , Y_{in} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



1. ADD R1, R2 ($R1 = R1 + R2$)

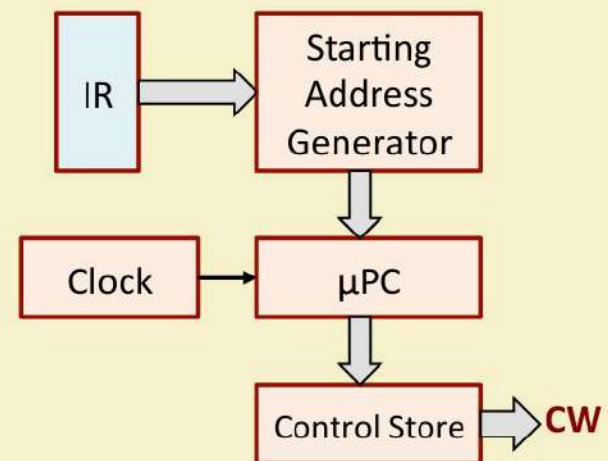
Steps	Action
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMFC
3	MDR_{out} , IR_{in}
4	$R1_{out}$, Y_{in}
5	$R2_{out}$, SelectY, Add, Z_{in}
6	Z_{out} , $R1_{in}$, End

2. ADD R1, LOCA ($R1 = R1 + \text{Mem}[\text{LOCA}]$)

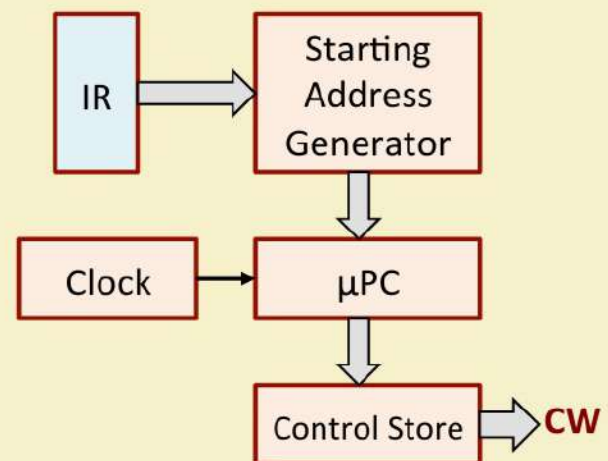
Steps	Action
1	$PC_{out}, MAR_{in}, \text{Read}, \text{Select4}, \text{Add}, Z_{in}$
2	$Z_{out}, PC_{in}, Y_{in}, \text{WMFC}$
3	MDR_{out}, IR_{in}
4	Address field of $IR_{out}, MAR_{in}, \text{Read}$
5	$R1_{out}, Y_{in}, \text{WMFC}$
6	$MDR_{out}, \text{SelectY}, \text{Add}, Z_{in}$
7	$Z_{out}, R1_{in}, \text{End}$

Microprogrammed Control Unit Design

- Control signals are generated by a program similar to machine language program.
- The *Control Store* (CS) stores the microroutines for all instructions of an ISA.
- The sequence of steps corresponding to the control sequence of a machine instruction is the *microroutine*.
- Each sequence of steps is a *control word* (CW) whose individual bits represent the various control signals.
- Individual control words in a microroutine are called *microinstructions*.



- Control-unit generates the control signals for an instruction by sequentially reading CWs of corresponding micro routine from CS.
- The μPC is used to read CWs sequentially from CS.
- Every time a new instruction is loaded into IR, output of Starting Address Generator is loaded into μPC .
- Then, μPC is automatically incremented by clock causing successive microinstructions to be read from CS.



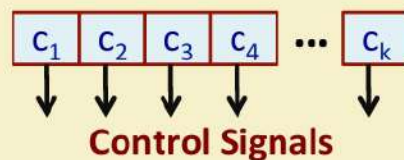
Control Store for “ADD R1, R2”

Micro-instr.	::	PC _{in}	PC _{out}	MAR _{in}	Read	MDR _{out}	IR _{in}	Y _{in}	Select	Add	Z _{in}	Z _{out}	R1 _{out}	R1 _{in}	R2 _{out}	WMFC	End	::
1	0	0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	0
2	0	1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	0
3	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0
5	0	0	0	0	0	0	0	0	1	1	1	0	0	0	1	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	0

Horizontal versus Vertical Microinstruction Encoding

- Broadly there are two alternate schemes to code the control signals in the control memory.
 - a) **Horizontal Micro-instruction Encoding**
 - Each control signal is represented by a bit in the micro-instruction.
 - Fewer control store words, with more bits per word.
 - b) **Vertical Micro-instruction Encoding**
 - Each control word represents a single micro-instruction in encoded form.
 - k-bit control word can support up to 2^k micro-instructions.
 - More control store words, with fewer bits per word

(a) Horizontal Micro-instruction Encoding



- Suppose that there are k control signals: $c_1, c_2, \dots, c_k$.
- In horizontal encoding, every control word stored in control memory (CM) consists of k bits, one bit for every control signal.
- Several bits in a control word can be 1:
 - Parallel activation of several micro-operations in a single time step.

0	1	0	1	1	0
---	---	---	---	---	---

 $\rightarrow$ c_2, c_4 and c_5 are activated together

(b) Vertical Micro-instruction Encoding

- Again consider that there are k control signals: $c_1, c_2, \dots, c_k$.
- We encode the control signals in a m -bit word in the control memory, where $k \leq 2^m$.
- Depending on the m -bit control word, exactly one control signal will be activated ($= 1$), while all others will remain de-activated ($= 0$).
 - At most one control signal can be activated in a time step.

